Software Engineering

# Assignment #04: Report

*Smart Campus Room Availability Checker*

**Student:** Shirinbek Shugyla, ID 24B032116

**Date:** 8 November 2026

**AI assistant used:** Claude (Sonnet 5.5)

**Versions:** the initial version is my Assignment #03 prototype (folder initial); the final version is in folder final.

## 1. Codebase Analysis

### 1.1 Context given to the AI

The AI received the Assignment #04 brief, my Assignment #03 code (a Python file with 26 unit tests and a React/TypeScript web page) and my Assignment #03 AI evidence file. It produced a repository map, the acceptance criteria and the project constraints, saved in final/docs/PROJECT_CONTEXT.md. The constraints were: no login, payments, live APIs or database; fictional data only; a fixed current time; no new packages.

### 1.2 Repository map of the initial version

| File | Lines | Content |
|---|---|---|
| src/App.tsx | 219 | State, derived values, three hint blocks, ResultCard, FieldInput, page layout |
| src/lib/availability.ts | 98 | Types, constants, validateStart / validateEnd / validateNow, canBook, minutesToHHMM |
| src/data/rooms.ts | 34 | Three rooms with id, name, blocked flag and bookings |
| Tests for the TypeScript code | 0 | None. The 26 Python tests cover a separate Python implementation. |

### 1.3 Maintainability risks

| # | Risk | File / function | Evidence |
|---|---|---|---|
| R1 | Three near-identical validators; the message "must be a whole number from 0 to 1440" exists three times | availability.ts: validateStart, validateEnd, validateNow | grep for the message: 3 matches; grep for isEmpty(raw): 3 matches |
| R2 | The whole-number pattern is copied four times | availability.ts line 24; App.tsx lines 59, 91, 95 | grep for the pattern: 4 matches |
| R3 | Silent fallback to the first room. With a catalog this could check another room's bookings. | App.tsx, useMemo: rooms.find(...) ?? rooms[0] | Reading the code |
| R4 | Mixed responsibilities: rules, validation, formatting and types in one file; state, formatting and two components in another | availability.ts (98 lines), App.tsx (219 lines) | Line counts and code reading |

| # | Risk | File / function | Evidence |
|---|---|---|---|
| R5 | Two almost identical JSX branches for the result card | App.tsx: ResultCard | grep: 2 blocks |
| R6 | No automated tests for the TypeScript rules | Whole src folder | Search for test files: 0 found |

### 1.4 Prioritised improvement plan (made before editing code)

1. **Safety net (R6).** Keep a verbatim copy of the initial rules as a test oracle and run AC1 to AC5 regression cases against it.
2. **New feature.** Extend the room data with capacity and equipment, and add pure filtering functions for F1 and F2.
3. **Single source of truth (R3, R4).** Move "form text to what the screen shows" into one pure function, so the selected room can never fall back to an unfiltered room.
4. **Remove duplication (R1, R2).** One generic validator and one whole-number pattern.
5. **Split the interface (R4, R5).** Small components; App.tsx keeps only state and layout.
6. **Verify.** Run all tests after each step and compare old and new behaviour with a differential test.

### 1.5 AI suggestions rejected or modified

**Rejected.** The AI first suggested Vitest, React Testing Library and jsdom for interface tests. I rejected this because it adds three packages while the project avoids extra packages. Node's built-in test runner is used for the logic, and one small render check uses packages that Vite and React already install.

**Modified.** The AI described the comparison test as covering "3,600+" inputs. The test failed on its own count check: the correct number is 2 x 3 x 24 x 24 = 3,456. The label and the assertion were corrected. All 3,456 comparisons between the old and new code passed.

## 2. Feature Extension

The prototype now has a catalog of five fictional rooms. Each room has an ID, a name, a capacity, an equipment list, a blocked status and its own bookings (minutes after midnight).

| ID | Name | Capacity | Equipment | Blocked | Bookings |
|---|---|---|---|---|---|
| R1 | Library Study Room | 6 | Whiteboard | No | 600-660 |
| R2 | Lab | 20 | Projector, Whiteboard, Computers | Yes | 600-660 |
| R3 | Seminar Room | 30 | Projector, Whiteboard, Microphone | No | 600-660, 720-780 |
| R4 | Collaboration Pod | 4 | Video conferencing | No | 840-900 |
| R5 | Lecture Hall | 120 | Projector, Microphone, Video conferencing | No | 660-780 |

**Filtering rules.** F1: room capacity must be at least the number of attendees. F2: the room must contain all selected equipment items. F3: a blocked room is never reported as available; blocked rooms stay in the list so the user sees the reason. F4: a clear message is shown when no room matches. F5: changing the selected room recomputes the result from that room's own bookings. The original rules AC1 to AC6 are unchanged.

### 2.1 Demonstration

Figures 1 to 4 show the running application. The table after them lists the scenarios as printed by the demo script (npm run demo).

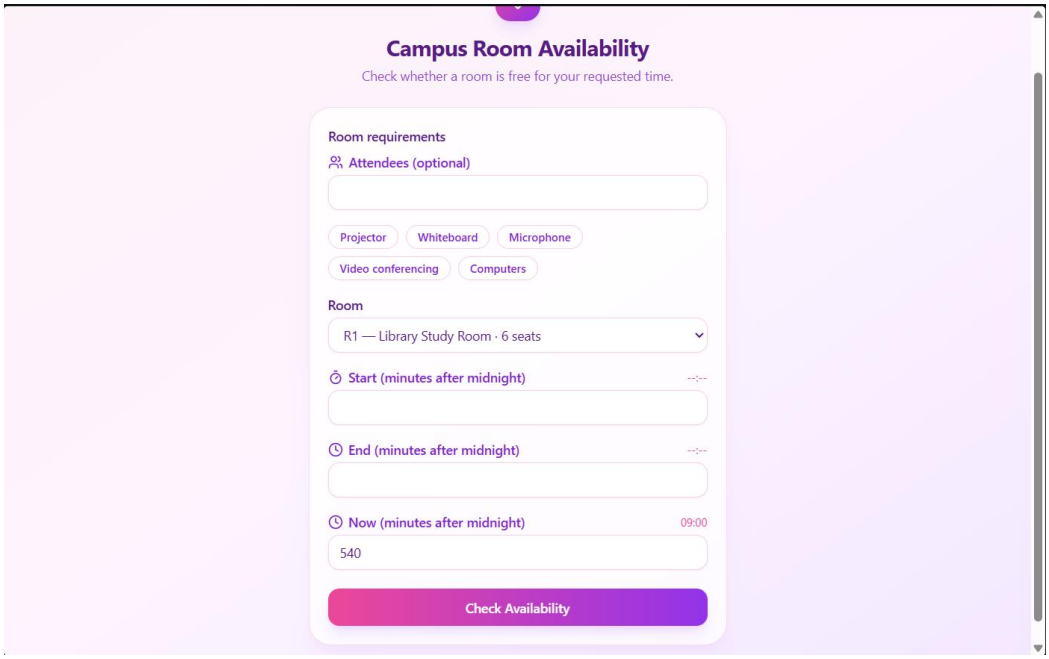



**Figure 1.** Initial screen: requirements block, room list and time fields.

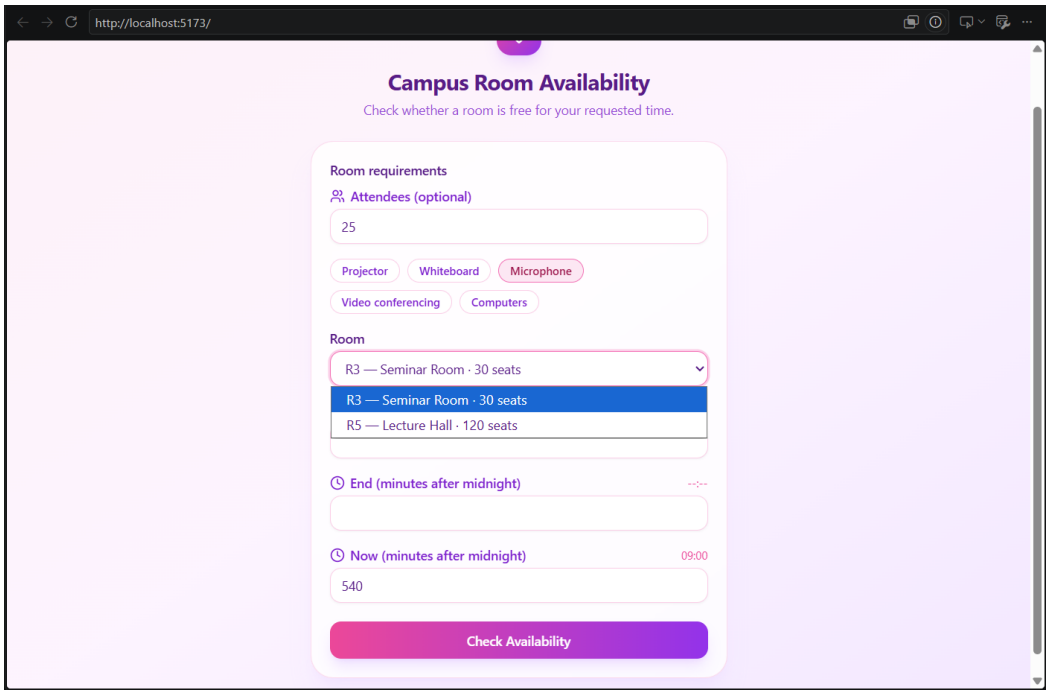



**Figure 2.** Successful filtering: 25 attendees and Microphone leave only R3 and R5 in the Room list.

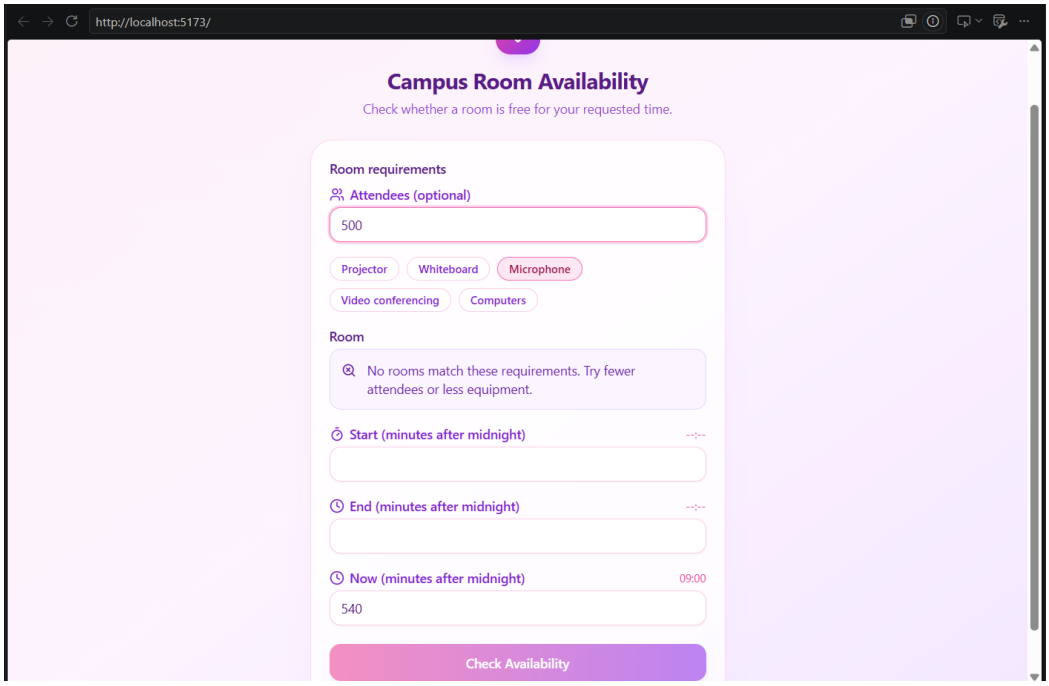



**Figure 3.** Unsuccessful filtering: 500 attendees match no room, so the empty-state message is shown and Check Availability is disabled.

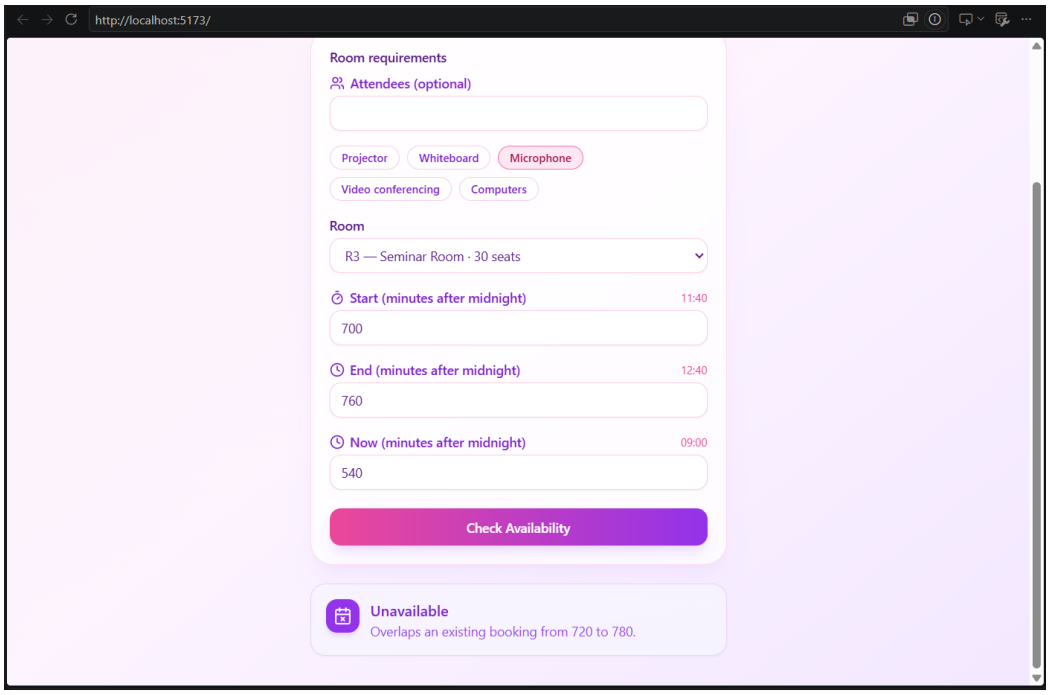



**Figure 4.** Room switching and availability check: R3, 700 to 760 is Unavailable because it overlaps the booking 720 to 780.

| Scenario | Rooms shown | Outcome |
|---|---|---|
| S1 25 attendees + Microphone; R3, 660-720 | R3, R5 | Available |
| S2 500 attendees | None | Empty state |
| S3 Projector + Video conferencing + Whiteboard | None | Empty state |
| S4 R1, 700-760 | All | Available |
| S5 same request, room switched to R3 | All | Unavailable: overlaps 720 to 780 |

| Scenario | Rooms shown | Outcome |
|---|---|---|
| S6 blocked room R2, free time slot | All | Unavailable: room is blocked |
| S7 AC2: 121 minutes in R1 | All | Unavailable: duration 121 exceeds 120 |
| S8 AC4: 660-720 touches booking 600-660 | All | Available |
| S9 AC4: 630-690 overlaps booking 600-660 | All | Unavailable: overlap |
| S10 AC1: start equals now | All | Unavailable: start must be after now |
| S11 AC6: empty start, text in end | All | Plain messages: "Start is required." and "End must be a whole number." |

## 3. Refactoring

The code was reorganised so that data, availability rules, filtering logic and interface are separate. Figure 5 shows the structure before and Figure 6 after the refactoring.

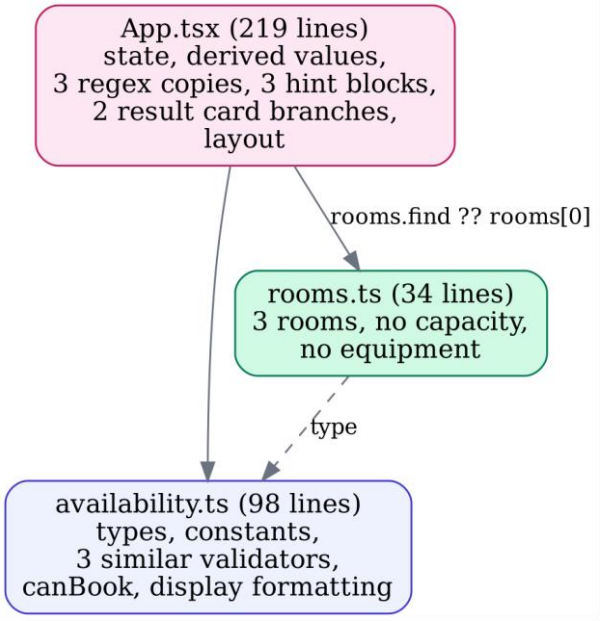



**Figure 5.** Before: three files with mixed responsibilities.

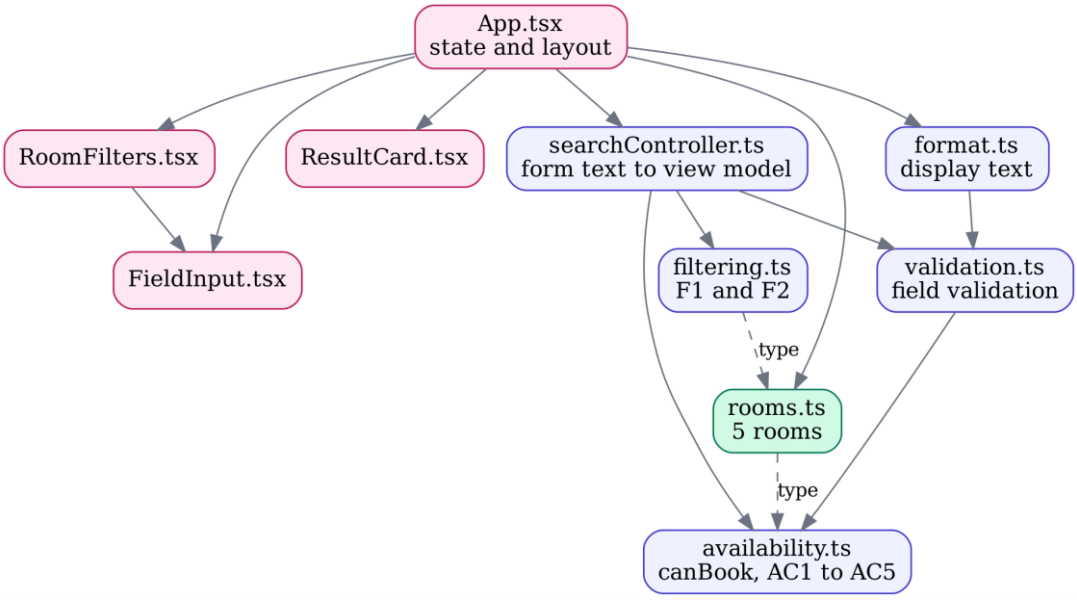



**Figure 6.** After: pink = interface, blue = logic without React, green = data. Ten modules, one responsibility each.

### 3.1 Design decisions

**Decision 1: a pure view-model function (searchController.ts).** It turns the raw form text into everything the screen shows: matching rooms, selected room, errors and result. The filters and validation can therefore be tested in plain Node without a browser, and the interface becomes a thin layer. The cost is one extra layer to read.

**Decision 2: the selected room comes only from the filtered list.** The old fallback to the first room was removed. If the chosen room is filtered out, the first matching room is used; if none match, no check is made. This enforces the rule that the check uses the chosen room's data and never another room's (risk R3, requirement F5).

**Decision 3: one generic validator with thin wrappers.** validateMinuteField replaces three copies. validateStart, validateEnd and validateNow remain as one-line wrappers, so callers and messages do not change. A comparison test proves identical results.

**Decision 4: canBook keeps its rule order and messages.** Only the parameter type became a read-only list, so the compiler forbids changing bookings (AC5). 3,456 comparisons show the behaviour equals the initial version.

## 4. Automated Testing

Commands, run in the final folder: **npm test** (Node 22.18 or newer) and **npm run test:ui**. The initial Python tests run with **python3 -m unittest test_booking**. Actual outputs are in test_results.txt.

| Test file | Tests | What it covers |
|---|---|---|
| regression_ac.test.ts | 16 | AC1 to AC5 on the initial and the final code (8 cases each): exactly 120 vs 121 minutes, touching vs overlapping intervals, start equal to now, 1440 bound, blocked room, unsorted bookings, no mutation |

| Test file | Tests | What it covers |
|---|---|---|
| filtering.test.ts | 9 | Catalog with at least 4 rooms; F1 exact vs insufficient capacity (6 vs 7, 120 vs 121); F2 all items; both filters; F3; F4; F5 (two tests); no mutation |
| controller_integration.test.ts | 6 | Integration: raw text, validation, filtering, availability; AC6 messages |
| refactor_equivalence.test.ts | 3 | Old vs new: 3,456 canBook inputs, 13 text samples through the validators, minutesToHHMM from -2 to 1442 |
| ui_smoke.mjs | 1 | Renders App.tsx: 5 room options, filters and form present, no result before Check |

**Result:** 34 of 34 Node tests passed; the interface render check passed; the Python baseline passed 26 of 26.

## 5. Quality Evaluation

| Measure | Initial | Final | How obtained |
|---|---|---|---|
| Automated tests, TypeScript code (passed / total) | 0 / 0 | 34 / 34 (+1 UI check) | Search for test files; npm test |
| Automated tests, Python rules | 26 / 26 | Not changed | python3 -m unittest |
| Whole-number pattern copies | 4 | 1 | grep over src |
| Near-identical validator bodies | 3 | 1 (+3 one-line wrappers) | grep for the range message: 3 to 1 |
| Separate hint blocks in App.tsx | 3 | 0 (one shared helper) | grep for startHint, endHint, nowHint |
| Result card JSX branches | 2 | 1 | grep over src |
| Source modules | 3 | 10 | File list |
| Largest source file | 219 lines | 130 lines | wc -l |
| Total source lines | 351 | 562 | wc -l (grew because the feature was added) |

The duplication rows are listed separately and are not added together, because the pattern copies and the hint blocks overlap.

**Responsibilities in the final structure.** rooms.ts: data. availability.ts: booking rules. validation.ts: text validation. filtering.ts: F1 and F2. format.ts: display text. searchController.ts: form to view model. components and App.tsx: presentation.

**Remaining limitations**

1. Typing and clicking in a real browser is not automated; only static rendering and the view-model logic are tested.
2. The Now field accepts 0 to 1440, as in the initial version, although the Assignment #03 brief says now is 0 to 1439. I kept the old behaviour on purpose; changing it would be a requirement change.

3. Equipment names are plain strings; a typo in rooms.ts is caught only by the catalog test, not by the compiler.
4. The Python and TypeScript versions still implement the same rules twice.